LexiPic Kiosk

Plugin — Technical Documentation

Version: 1.2.1

Type: WordPress plugin (text domain: lexipic-kiosk)

Requires: WordPress 6.0+, PHP 7.4+

Publisher: BhoomiTech Heritage and Development Foundation

Part of: LexiPic Suite — see "LexiPic Suite Overview" for cross-module context

Table of Contents

Table of Contents 2
1. Introduction & Scope 4
1.1 Key capabilities 4
2. System Requirements 5
3. Architecture Overview 6
3.1 File structure 6
3.2 Component responsibilities 6
3.3 Request lifecycle 7
4. Database Schema 8
4.1 wp_lexipic_sets 8
4.2 wp_lexipic_entries 8
4.3 Schema versioning and the 1.1.0 media migration 9
5. REST API Reference 10
5.1 Endpoint summary 10
5.2 Permission model in detail 11
5.3 Sets endpoints in detail 11
GET /sets 11
POST /sets (import) 11
DELETE /sets/{slug}, POST .../rename, .../active, /sets/reorder 11
5.4 Backup, restore, flush 11
6. Media Storage Strategy 13
6.1 Storage layout 13
6.2 Conversion flow (maybe_save()) 13
6.3 Cleanup and safety checks 13
7. PIN & Session Security Model 14
7.1 Setting and storing the PIN 14
7.2 Verifying a PIN (POST /pin/verify) 14
7.3 Session tokens 14
8. Import Normalizer 15
8.1 Accepted shapes 15
8.2 Field mapping and validation 15
9. Front-End Architecture 16
9.1 Display surfaces 16
9.2 app.js: server-first reads with offline fallback 16
9.3 Idle timer & autoplay 17
9.4 On-screen PIN-gated admin panel 17

10. Admin Screens 18
11. Capabilities & Permissions 19
12. Allowlist Subsystem (PWA Gatekeeping) 20
12.1 Why a separate option 20
12.2 Domain normalization 20
12.3 No separate PIN for the PWA 20
13. Settings Reference 21
14. Installation & Configuration 22
15. Uninstall Behavior 23
16. Operational Notes 24
17. Changelog 25

1. Introduction & Scope

LexiPic Kiosk is the distribution and central-management module of the LexiPic Suite. It is an independent WordPress plugin — entirely separate from the LexiPic core plugin's database tables, REST namespace, and codebase — purpose-built for displaying word sets on touchscreen kiosks and for managing those kiosks from a single place, including kiosks that aren't running WordPress at all (via the companion PWA in Remote URL mode).

This document covers the plugin's internal architecture, database schema (including its media-storage migration history), REST API, security model, and front-end behaviour. For how this module fits alongside LexiPic (core) and the LexiPic Kiosk PWA, see the companion "LexiPic Suite Overview" document.

1.1 Key capabilities

- Word-set management from wp-admin: import a .json export, rename, reorder, activate/deactivate, or delete sets.
- Two display modes: the [lexipic_kiosk] shortcode for inline embedding on any page, or a dedicated fullscreen kiosk URL with no theme header, footer, or admin toolbar.
- A PIN-protected on-screen admin panel, letting a kiosk operator manage sets directly from the touchscreen without a WordPress login.
- Its own database tables, entirely separate from LexiPic's and from the WordPress Media Library; images and audio are stored as ordinary files under wp-content/uploads/lexipic-kiosk/.
- Local IndexedDB caching on the kiosk device so it keeps working through a brief network interruption, while WordPress remains the single source of truth.
- Full backup/restore of the entire kiosk dataset as a single JSON file, from either wp-admin or the on-screen panel.
- An approved-domain allowlist consumed by the companion standalone LexiPic Kiosk PWA, so a kiosk device running the PWA can only ever be pointed at sites explicitly vetted from wp-admin.

2. System Requirements

Requirement	Value
WordPress	6.0 or later (tested up to 6.5)
PHP	7.4 or later
PHP extension	None required beyond standard WordPress/PHP facilities.
Kiosk device browser	Any modern browser for the embedded shortcode/fullscreen page. IndexedDB is required for offline-resilient local caching (universally available in modern browsers).

3. Architecture Overview

Unlike LexiPic core, LexiPic Kiosk uses a lightweight PSR-4-lite autoloader (registered in the main plugin file) so class files don't need to be required individually. Class file naming follows the WordPress convention class-lexipic-kiosk-{name}.php defining Lexipic_Kiosk_{Name}, searched first under includes/ then admin/.

3.1 File structure

```
lexipic-kiosk/
├── lexipic-kiosk.php           Bootstrap: constants, autoloader, activation
├── uninstall.php               Conditional cleanup (opt-in only)
├── readme.txt
├── includes/
│   ├── class-lexipic-kiosk.php             Central orchestrator
│   ├── class-lexipic-kiosk-db.php          Schema + migration
│   ├── class-lexipic-kiosk-set-repository.php  Sets/entries data access
│   ├── class-lexipic-kiosk-media.php       Base64 <-> file conversion
│   ├── class-lexipic-kiosk-rest-controller.php REST namespace
│   ├── class-lexipic-kiosk-settings.php    Idle/autoplay/PIN settings
│   ├── class-lexipic-kiosk-session.php     PIN session tokens + rate limit
│   ├── class-lexipic-kiosk-allowlist.php   PWA domain allowlist
│   ├── class-lexipic-kiosk-capabilities.php  manage_lexipic_kiosk cap
│   ├── class-lexipic-kiosk-import-normalizer.php  Import shape handling
│   ├── class-lexipic-kiosk-shortcode.php   [lexipic_kiosk]
│   └── class-lexipic-kiosk-template.php    Fullscreen kiosk URL
├── admin/
│   ├── class-lexipic-kiosk-admin.php       wp-admin screens + handlers
│   └── views/                              sets-page, settings-page,
│                                            embed-page, allowlist-page
└── public/
    ├── css/kiosk.css
    ├── views/kiosk-markup.php              Shared kiosk DOM
    ├── views/kiosk-fullscreen.php          Bare HTML document
    └── js/
        ├── app.js                          Kiosk front-end + cache
        └── admin-ui.js                     On-screen PIN/admin accordion
```

3.2 Component responsibilities

Component	Responsibility
Lexipic_Kiosk	Central orchestrator, instantiated once on plugins_loaded. Wires the REST controller, settings, shortcode, fullscreen template, and conditionally loads the admin screens (is_admin()) and public assets (only on pages that need them).
Lexipic_Kiosk_DB	Owns the two custom tables and schema versioning, including a one-time data migration for sites upgrading from the original inline-base64 storage format (§4.3).
Lexipic_Kiosk_Set_Repository	All sets/entries data access. Method names intentionally mirror the original standalone app's IndexedDB function names (saveSet, getAllSets, etc.) so REST and front-end logic map onto it 1:1.
Lexipic_Kiosk_Media	Converts inline base64 data URIs (the import/export wire format)

Component	Responsibility
	into real files under wp-content/uploads/lexipic-kiosk/, and cleans up orphaned files on delete/replace.
Lexipic_Kiosk_REST_Controller	Registers and implements every route under lexipic-kiosk/v1, including the dual logged-in-user-OR-kiosk-PIN-token permission model.
Lexipic_Kiosk_Settings	Idle-timer and autoplay defaults, and kiosk PIN storage (hashed, never stored or transmitted in plain form after being set).
Lexipic_Kiosk_Session	Issues and verifies short-lived kiosk session tokens (WordPress transients) after a successful PIN check; rate-limits PIN attempts.
Lexipic_Kiosk_Allowlist	Stores and normalizes the list of domains the standalone PWA is permitted to connect to in Remote URL mode.
Lexipic_Kiosk_Capabilities	Defines and grants the manage_lexipic_kiosk capability.
Lexipic_Kiosk_Import_Normalizer	Accepts both the current plugin export shape and the legacy flat-array PWA prototype shape, validating and size-capping embedded media.
Lexipic_Kiosk_Shortcode / Template	Render the kiosk: [lexipic_kiosk] for inline embedding, or a dedicated fullscreen URL with admin-toolbar suppression.
public/js/app.js	The kiosk's own front-end application: server-first reads with IndexedDB fallback, the tile grid / card slider, the idle timer, and the on-screen PIN-gated admin accordion.

3.3 Request lifecycle

1. Lexipic_Kiosk::maybe_enqueue_public_assets(), hooked to wp_enqueue_scripts, checks whether the current request is the fullscreen kiosk template or a page whose content contains the [lexipic_kiosk] shortcode, and only then enqueues the (fairly heavy) kiosk JS/CSS — Lexipic_Kiosk_Assets::enqueue_public() is idempotent, so it is also safely called defensively from the shortcode handler itself for builder/late-render edge cases.
2. public/js/app.js boots, reading window.LexipicKioskConfig (REST base URL + nonce, injected via wp_localize_script) and calling GET /sets to populate the tile grid, with an IndexedDB-cached fallback if that request fails.
3. Opening a set fetches GET /sets/{slug} (full entries this time) and caches that specific set's entries locally, so re-opening it later works even offline.
4. Every write (import, rename, reorder, toggle-active, delete, settings, allowlist, backup/restore, flush) goes straight to the REST API; the local IndexedDB cache is only ever updated as a side effect of a successful network read or write, never treated as authoritative on its own.

4. Database Schema

LexiPic Kiosk creates two custom tables, kept fully separate from WordPress posts/postmeta and from the Media Library by design (Lexipic_Kiosk_DB::install(), via dbDelta()).

4.1 wp_lexipic_sets

Column	Type	Notes
id	BIGINT UNSIGNED, PK, AUTO_INCREMENT	
slug	VARCHAR(191) NOT NULL, UNIQUE	
name	VARCHAR(255) NOT NULL	
language	VARCHAR(20) NOT NULL DEFAULT ''	Language code, e.g. bpm.
language_label	VARCHAR(120) NOT NULL DEFAULT ''	Display label, e.g. "Bishnupriya Manipuri".
active	TINYINT(1) NOT NULL DEFAULT 1	Whether the set is shown on the kiosk.
set_order	BIGINT NOT NULL DEFAULT 0	Display order, indexed together with active.
created_at / updated_at	DATETIME NOT NULL	

4.2 wp_lexipic_entries

Column	Type	Notes
id	BIGINT UNSIGNED, PK, AUTO_INCREMENT	
set_id	BIGINT UNSIGNED NOT NULL	Indexed alone and together with entry_order.
entry_order	BIGINT NOT NULL DEFAULT 0	
word_script / word_roman / description	TEXT NOT NULL	
image / audio	TEXT NOT NULL	Public URL (post-1.1.0) under wp-content/uploads/lexipic-kiosk/, not a blob.
image_mime / audio_mime	VARCHAR(60) NOT NULL DEFAULT ''	

4.3 Schema versioning and the 1.1.0 media migration

Lexipic_Kiosk_DB::DB_VERSION is checked against a stored option (lexipic_kiosk_db_version) on every admin_init, so the schema self-heals after a plugin file update without requiring a manual deactivate/reactivate cycle (maybe_upgrade()).

Prior to version 1.1.0, image and audio data was stored as inline base64 directly in LONGTEXT columns. As of 1.1.0, both fields store a short file URL instead, with the actual binary written under wp-content/uploads/lexipic-kiosk/{set_id}/. On any site still running the older schema, install() calls migrate_inline_media_to_files() before dbDelta() narrows the column type to TEXT — this walks every entry whose image/audio column still starts with data:, converts it via Lexipic_Kiosk_Media, and updates the row, in batches of 50 to avoid a single-request timeout on a large existing library. The migration is idempotent and a no-op on a fresh install or an already-migrated site.

> **Why this mattered:** Narrowing a LONGTEXT column holding an unconverted long base64 string to TEXT risks silent truncation by MySQL on a host with strict mode disabled, which would corrupt that entry's media. Converting to files first, before the column is narrowed, avoids that failure mode entirely.

The .json import/export wire format is unaffected by this internal storage change — a set exported before or after 1.1.0 round-trips identically; only the database's internal representation changed, and backups produced post-1.1.0 are dramatically smaller as a direct result.

5. REST API Reference

All routes are registered under the namespace lexipic-kiosk/v1. "Protected" routes accept either a logged-in user with the manage_lexipic_kiosk capability, or a valid kiosk session token (obtained from /pin/verify) passed as the X-Lexipic-Kiosk-Token header — see §7 for the security model in detail.

5.1 Endpoint summary

Method	Route	Access	Purpose
GET	/sets	Public	List sets — lightweight metadata + entryCount by default; ?include_entries=1 for full entries.
POST	/sets	Protected	Create or overwrite a set (import) by slug.
POST	/sets/reorder	Protected	Swap two sets' display order. Params: slug_a, slug_b.
GET	/sets/{slug}	Public	Get one set with its full entries.
DELETE	/sets/{slug}	Protected	Delete a set, its entries, and its media files.
POST	/sets/{slug}/rename	Protected	Rename a set (slug is recomputed; collisions rejected).
POST	/sets/{slug}/active	Protected	Toggle a set's active flag.
GET	/settings	Public	Idle timer + autoplay defaults, plus pinConfigured (never the PIN itself).
POST	/settings	Protected — admin login only	Update site-wide idle/autoplay defaults.
POST	/pin/verify	Public	Exchange a PIN for a short-lived kiosk session token. Rate-limited.
GET	/allowlist	Public	Approved domain list for the standalone PWA's Remote URL mode. No PIN is returned.
POST	/allowlist	Protected — admin login only	Replace the approved domain list.
GET	/backup	Protected	Full export payload: every set (with entries) + settings.
POST	/restore	Protected	Destructive full restore from a backup payload.
POST	/flush	Protected	Wipe all sets, entries, and media files.

5.2 Permission model in detail

Callback	Logic
check_manage_permission()	True if current_user_can('manage_lexipic_kiosk'), OR the request carries a valid X-Lexipic-Kiosk-Token verified by Lexipic_Kiosk_Session::verify_token(). Used for ordinary set management (create, rename, reorder, toggle-active, delete) and backup/restore/flush — actions a PIN-holding kiosk operator should be able to perform locally.
check_admin_login_permission()	True only if current_user_can('manage_lexipic_kiosk') — a kiosk PIN session is explicitly not sufficient. Used for the site-wide settings defaults and the PWA allowlist, since both affect every kiosk device, not just the one where the PIN was entered.

5.3 Sets endpoints in detail

GET /sets

Returns set metadata plus a per-set entryCount by default — deliberately not every entry's full data, since pulling that in unconditionally is what made the original response balloon with multiple 100+-entry sets. Pass ?include_entries=1 (used by the admin sets list and by /backup internally) to get every set's full entries array in one batched query instead of one query per set.

POST /sets (import)

Accepts a JSON body in either the current plugin format or the legacy flat-array format; Lexipic_Kiosk_Import_Normalizer::process() (see §8) normalizes it first. If a set with the same slug already exists, save_set() wholesale-replaces its entries and metadata while preserving its active flag and set_order position — re-importing the same export overwrites rather than duplicates, matching the original app's behaviour. The previous entry list's media files are deleted first (Lexipic_Kiosk_Media::delete_for_set()) so re-imports never leak orphaned files, and the delete-then-insert runs inside a single database transaction so a failure partway through can't leave a set with a half-replaced word list.

DELETE /sets/{slug}, POST .../rename, .../active, /sets/reorder

- Delete removes the set, its entries, and every media file under its set-specific upload subfolder.
- Rename recomputes the slug from the new name and rejects on collision with a different existing set; unlike the original IndexedDB version (where slug was the primary key, requiring a delete+reinsert to rename), this is a plain UPDATE.
- Active toggling and reordering re-read current state from the database rather than trusting caller-supplied order values, so rapid repeated calls (e.g. fast double-clicks on move-up/down) can't desync stored order from what was last displayed.

5.4 Backup, restore, flush

GET /backup returns { backupFormatVersion: 1, exportedAt, sets: [...with entries...], settings }. POST /restore validates that the payload has a sets array and that every set has a valid slug before touching the database, then calls flush_all() and re-inserts everything from the payload — this is fully destructive to whatever was previously on the kiosk. POST /flush performs the same wipe (sets,

entries, and every set’s media files) without a subsequent restore, used by the admin’s "Flush System Database" action.

6. Media Storage Strategy

Lexipic_Kiosk_Media converts inline base64 data URIs — the format used on the wire for import/export and produced natively by the original standalone prototype — into real files on disk, so the database and every REST response carry a short URL string instead of a multi-hundred-kilobyte blob per image/audio field. The import/export JSON format itself is unchanged by this; only the internal storage representation differs.

6.1 Storage layout

```
wp-content/uploads/lexipic-kiosk/
└── {set_id}/
    ├── {random-20-char-filename}.jpg
    ├── {random-20-char-filename}.webm
    ├── ...
    └── index.php          (empty stub — directory listing protection)
```

Filenames are generated via wp_generate_password(20, false, false) — random, not derived from the original filename — with the extension chosen from a MIME-to-extension map (EXTENSIONS) covering common image and audio types, falling back to deriving an extension from the MIME subtype for anything unrecognized.

6.2 Conversion flow (maybe_save())

5. If the supplied value is not a data: URI (i.e. it's already a URL, or empty), it is returned unchanged — this makes maybe_save() safe to call unconditionally on every entry without double-processing already-converted media.
6. A data: URI is parsed via regex into its MIME type and base64 payload; a non-base64 data URI (this plugin only ever produces/accepts base64) is rejected as malformed.
7. The decoded binary is written to the set's upload subfolder (created on first use, with an empty index.php stub for directory-listing protection) under a randomly generated filename, and the resulting public URL is returned.

6.3 Cleanup and safety checks

- delete_for_set(set_id) removes every file in a set's media subfolder and the folder itself — called whenever a set's entries are wholesale-replaced (re-import) or the set is deleted, so re-imports and deletions never leak orphaned files.
- **delete_url(url)** removes a single previously-saved file given the URL stored in an entry's image/audio column. It first checks the URL starts with this site's own uploads base URL for the lexipic-kiosk subdirectory, then re-validates the resolved path via realpath() stays within that subdirectory — defense-in-depth against a malformed or legacy URL containing a path-traversal sequence. A URL that isn't recognized as this plugin's own is left untouched.
- **looks_like_own_url(value)** used by the import normalizer to decide whether an already-converted (non-data-URI) value should be accepted as-is. It deliberately checks only for the /lexipic-kiosk/ path segment rather than an exact match against the current site's uploads base URL, so a backup restored onto a different domain (or a site migrated to a new URL) still recognizes its own previously-exported media links. The stricter realpath() check in delete_url() means this looser acceptance check never widens what can actually be deleted.

7. PIN & Session Security Model

The kiosk PIN replaces what was, in the original standalone prototype, a hardcoded client-side string comparison (visible in the page source, with no attempt limit at all). It now gates access through a dedicated, rate-limited, server-verified REST endpoint, independent of any WordPress login.

7.1 Setting and storing the PIN

An administrator sets the PIN from LexiPic Kiosk → Settings. Lexipic_Kiosk_Settings::set_pin() hashes it with wp_hash_password() before storing — the plain-text value is never persisted. No default PIN is seeded on activation; until an administrator explicitly sets one, has_pin() is false and the on-screen admin panel stays locked entirely, rather than shipping a guessable factory default.

7.2 Verifying a PIN (POST /pin/verify)

8. The requester's IP is checked against a rate limit (5 attempts per 60-second window, per IP) before the PIN itself is even compared; a request over the limit receives 429 Too Many Requests without revealing anything about whether a PIN is configured.
9. The submitted PIN is checked against the stored hash via wp_check_password(). If no PIN has ever been configured, verification fails closed (never open) regardless of what was submitted.
10. A successful check clears that IP's failure count (so an earlier mistyped attempt doesn't compound against a later correct one) and issues a new session token via Lexipic_Kiosk_Session::issue_token().
11. A failed check increments that IP's failure count, persisted in a WordPress transient keyed by an MD5 hash of the IP, expiring after the 60-second window.

7.3 Session tokens

Property	Value
Generation	wp_generate_password(48, false, false) — a random opaque string, not a JWT or otherwise self-describing credential.
Storage	A WordPress transient keyed by the token itself; the transient's mere presence is the proof of validity, so no secret material needs to round-trip on verification.
Lifetime	1800 seconds (30 minutes) — chosen with headroom above the kiosk's own idle-timeout ceiling (300 seconds max on the on-screen slider), so a long admin session doesn't expire mid-task.
Transport	Sent by the client as the X-Lexipic-Kiosk-Token header on every protected write call.
Client-side storage	Held only in a JavaScript variable for the lifetime of the page — deliberately never written to localStorage, sessionStorage, or a cookie, so a shared or idle kiosk can't leak a still-valid admin session across a reload or to another browser tab.
Revocation	revoke_token() deletes the transient immediately; called when the kiosk's own idle timer fires, so a stolen or leftover token can't be replayed after the UI has already re-locked.

8. Import Normalizer

Lexipic_Kiosk_Import_Normalizer::process() accepts an arbitrary decoded JSON body and produces the shape Lexipic_Kiosk_Set_Repository::save_set() expects, mirroring the original standalone app's LexipicNormalizer module exactly so behaviour is unchanged across the port to WordPress.

8.1 Accepted shapes

- **Current plugin format** { set: { name, slug, language }, entries: [...] } — set metadata is read from the set object, falling back to sensible defaults (auto-generated name/slug, language bpm) for any missing field.
- **Legacy flat-array format** a bare JSON array of entries with no set wrapper at all — treated as the entries list directly, with an auto-generated set name ("Imported Set {date}") and the bpm language assumed.
- **Anything else** raises an exception ("This file doesn't look like a LexiPic export."), surfaced to the importer as a clear error rather than a generic failure.

8.2 Field mapping and validation

Output field	Source
word_script	entry.word_script, falling back to the legacy key entry.heritage
word_roman	entry.word_roman, falling back to the legacy key entry.transliteration
description	entry.description
image / audio	Validated via validate_data_uri() — see below

An entry with no script word, no Roman word, and no image is dropped entirely (matching the original's e.word_script || e.word_roman || e.image filter) — a fully empty row is never imported.

validate_data_uri() handles a value that may, by this point in the plugin's life, be in either of two shapes:

- A raw base64 data URI — size-capped (the decoded payload, estimated from base64 length without a full decode, must be under roughly 8MB) and checked for well-formedness here; the actual decode-to-file conversion happens later, once a set_id exists (see §6.2). An oversized or malformed data URI is silently dropped (empty string) rather than failing the whole import, so one bad card doesn't block an otherwise-valid set.
- An already-converted URL — accepted as-is only if Lexipic_Kiosk_Media::looks_like_own_url() recognizes it as one of this plugin's own previously-exported media links; any other foreign URL is dropped.

9. Front-End Architecture

9.1 Display surfaces

Surface	How it's rendered
[lexipic_kiosk] shortcode	Lexipic_Kiosk_Shortcode::render() wraps public/views/kiosk-markup.php in a height-configurable , embedding the kiosk inline on a normal themed page. Accepts a height attribute, e.g. [lexipic_kiosk height="800px"].
Fullscreen kiosk URL	Lexipic_Kiosk_Template registers a rewrite rule for a filterable slug (kiosk by default, via the lexipic_kiosk_template_slug filter) and short-circuits the normal template hierarchy to print public/views/kiosk-fullscreen.php — a bare HTML document with no theme header/footer/sidebar, though wp_head() / wp_footer() are still called so other enqueued assets (including this plugin's own) print correctly. Also reachable without pretty permalinks via /?lexipic_kiosk=1.

> **Security note:** The fullscreen kiosk URL explicitly suppresses the WordPress admin toolbar via show_admin_bar(false), called on after_setup_theme (early enough to reliably take effect; template_redirect runs too late for this specific WordPress API). This matters because a public kiosk display must never expose "Howdy, {name}" or a one-click link into wp-admin just because an administrator happens to still be logged in on that browser from earlier testing. This fix shipped in version 1.2.1 — see §17.

9.2 app.js: server-first reads with offline fallback

The kiosk's front-end script treats WordPress (via the REST API) as the authoritative source of truth, with IndexedDB demoted to a read-through local cache rather than its own source of data — a deliberate architectural shift from the original standalone app, where IndexedDB was authoritative.

IndexedDB store	Purpose
sets	Lightweight metadata for every set (name/slug/order/active/entryCount, no entries) — refreshed on every successful /sets read; powers the offline home-screen tile grid.
set-entries	One set's full entries at a time, keyed by slug — refreshed whenever that specific set is opened; powers offline re-opening of whichever set(s) were last viewed on this device. A set never opened on this device while online simply isn't available offline.
settings	The kiosk's idle-timer/autoplay defaults, cached the same way.

The read pattern (getAllSets(), getSetWithEntries(slug), getKioskSettings()) is consistent throughout: try the REST API first; on success, refresh the corresponding IndexedDB cache and return the fresh data; on any network failure, fall back to whatever was last cached. Every write (import, rename, delete, settings, etc.) goes straight to the REST API — the cache is never written to directly outside of these read-refresh paths.

9.3 Idle timer & autoplay

The kiosk auto-returns to the sets grid after a configurable idle period (default 90 seconds) with no touch/scroll/click activity, and can optionally autoplay each word's audio when its card opens. Both have a site-wide default (set from wp-admin, requiring an actual admin login per §7) and a per-device override (adjustable from the on-screen accordion once unlocked with the PIN, stored only in that device's local IndexedDB settings cache) — a kiosk operator's own tweaks never silently become the site-wide default for every other kiosk.

9.4 On-screen PIN-gated admin panel

A password-style input in the kiosk header, combined with admin-ui.js and app.js, lets an operator unlock an accordion panel offering import, rename, reorder, activate/deactivate, delete, backup/restore, and flush — every action available from wp-admin's Word Sets screen, available directly at the kiosk. Submitting the correct PIN calls POST /pin/verify; the returned token is held in memory only (kioskSessionToken, never persisted — see §7.3) and attached to every subsequent protected REST call for the remainder of that page load.

10. Admin Screens

Lexipic_Kiosk_Admin registers a top-level "LexiPic Kiosk" wp-admin menu, gated by the manage_lexipic_kiosk capability rather than a CPT list table, since sets live in custom tables rather than wp_posts.

Screen	Purpose
Word Sets	The default landing page — a table of every set (name, slug, language, word count, active toggle, reorder arrows, delete) plus a JSON import form.
Settings	Kiosk Admin PIN (set/change/clear), default idle auto-close timer, default autoplay audio, and the "Delete all data on uninstall" opt-in checkbox.
Embed & Kiosk URL	Read-only reference: the [lexipic_kiosk] shortcode (with a height-customization example) and both the pretty-permalink and plain-query-string forms of the fullscreen kiosk URL.
Approved Kiosk URLs	The PWA domain allowlist textarea (one hostname per line) — see §12.

All mutating actions (save settings, save allowlist, import, delete, toggle-active, reorder, export backup) are handled via admin-post.php hooks, each independently re-checking the manage_lexipic_kiosk capability and a matching WordPress nonce (verify_request()) before doing anything, regardless of what the requesting page displayed.

11. Capabilities & Permissions

Mechanism	Detail
manage_lexipic_kiosk capability	Defined by Lexipic_Kiosk_Capabilities and granted to the Administrator role automatically on activation. Using a dedicated capability rather than checking for manage_options means a site owner can later grant kiosk management to an Editor or a custom role without handing them full site administration.
Kiosk PIN	Independent of any WordPress account — see §7. Lets a non-WordPress-user (an event volunteer, a kiosk-site staff member) manage sets locally without ever logging into wp-admin.
Public read endpoints	GET /sets, GET /sets/{slug}, GET /settings, and GET /allowlist require no authentication at all — the kiosk display itself, and the PWA's allowlist check, must work for anonymous visitors/devices.

12. Allowlist Subsystem (PWA Gatekeeping)

LexiPic Kiosk → Approved Kiosk URLs is relevant only to the separate, standalone LexiPic Kiosk PWA — a browser-only kiosk app (see the companion "LexiPic Kiosk PWA — Technical Documentation") that can connect to a "Remote URL" instead of needing WordPress installed on the kiosk device itself. It has no effect on this site's own embedded shortcode or fullscreen kiosk.

12.1 Why a separate option

Lexipic_Kiosk_Allowlist is deliberately a separate, simple option (lexipic_kiosk_url_allowlist) rather than folded into Lexipic_Kiosk_Settings — it has nothing to do with this site's own kiosk behaviour, only with vetting which other kiosk endpoints a PWA instance may be pointed at. Keeping it isolated also means a site that only runs the WordPress-embedded kiosk, and never serves PWA allowlist requests, pays no extra cost.

The list is empty by default on activation — an administrator must explicitly approve at least one domain before any PWA can be pointed anywhere, rather than shipping a permissive default.

12.2 Domain normalization

save_from_textarea() accepts messy admin input — "https://example.com/", "example.com", " EXAMPLE.com " — one entry per line, and reduces each to a bare lowercase hostname matching exactly what PHP's parse_url() or the PWA's own new URL().hostname would produce for a real request. A scheme is prepended if missing (so parse_url() can reliably find the host component at all), then the host is extracted and lowercased. Subdomains are not automatically included — kiosk.example.com must be listed separately from example.com if both should be permitted.

12.3 No separate PIN for the PWA

The allowlist endpoint (GET /allowlist) never returns a PIN or PIN hash in any form. The PWA's "change source" gate calls this plugin's existing POST /pin/verify endpoint directly — the same hashed, rate-limited PIN already used for this site's own on-screen kiosk admin panel. There is no second, weaker, PWA-specific credential to manage.

> **Version note:** The Approved Kiosk URLs page and the public /allowlist endpoint were introduced in version 1.2.0, reusing the existing /pin/verify endpoint rather than introducing a second PIN — see §17.

13. Settings Reference

Stored in the lexipic_kiosk_settings WordPress option:

Key	Type	Default	Description
idle_time_seconds	integer	90	Site-wide default idle-timeout a newly-loaded kiosk starts with.
autoplay_audio	boolean	false	Site-wide default for whether a card's audio plays automatically when opened.
pin_hash	string (hashed)	'' (unset)	wp_hash_password() output; empty means the on-screen admin panel is inaccessible.
delete_on_uninstall	boolean	false	Opt-in flag controlling whether deleting the plugin also deletes all data — see §15.

The PWA domain allowlist is stored separately as lexipic_kiosk_url_allowlist (see §12); a record of pending kiosk PIN rate-limit counters and active session tokens lives only in short-lived WordPress transients (see §7), not in this option.

14. Installation & Configuration

12. Upload the plugin folder to wp-content/plugins/, or install the zip via Plugins → Add New → Upload Plugin.
13. Activate the plugin. lexipic_kiosk_activate() installs the database tables, seeds default settings (no PIN yet) and an empty allowlist, grants manage_lexipic_kiosk to Administrators, and flushes rewrite rules so the fullscreen kiosk URL's pretty-permalink form works immediately.
14. Go to LexiPic Kiosk → Settings and set a PIN — the on-screen admin panel stays locked until one is configured.
15. Go to LexiPic Kiosk → Word Sets and import a first .json export (produced by LexiPic core, or by this plugin's own /backup).
16. Either add [lexipic_kiosk] to a page, or visit the dedicated kiosk URL shown under LexiPic Kiosk → Embed & Kiosk URL.
17. (Optional) If deploying the standalone PWA in Remote URL mode anywhere, go to LexiPic Kiosk → Approved Kiosk URLs and add the domain(s) it should be permitted to display.

15. Uninstall Behavior

Deactivating the plugin never touches data — flush_rewrite_rules() only. Deleting the plugin (uninstall.php) is conditional, unlike LexiPic core's unconditional cleanup:

- If "Delete all data on uninstall" was checked on the Settings screen beforehand, deletion drops both database tables, removes the manage_lexipic_kiosk capability from every role that has it, and deletes the settings and allowlist options.
- If left unchecked (the default), everything is left in place — reinstalling the plugin later picks up right where it left off, with all sets, settings, and the allowlist intact.

> **Note:** Media files under wp-content/uploads/lexipic-kiosk/ are not explicitly removed by uninstall.php in either case; they remain on disk unless removed manually or via the per-set deletion path during normal use.

16. Operational Notes

- A kiosk device that has never successfully loaded a given set while online has no offline fallback for that set — IndexedDB can only cache what it has actually fetched at least once.
- The kiosk PIN is a single shared secret for the whole site’s kiosk fleet, not a per-device or per-operator credential — anyone who knows it can manage sets from any kiosk’s touchscreen or via the PWA’s "change source" gate.
- Approved Kiosk URLs only governs the standalone PWA’s Remote URL mode; it has no bearing on who can view this site’s own [lexipic_kiosk] embed or fullscreen page, which follow ordinary WordPress page visibility.
- Schema self-healing (maybe_upgrade()) runs on admin_init, so a version bump takes effect the next time any logged-in user with appropriate access visits wp-admin — not immediately on file deployment alone.

17. Changelog

Version	Notes
1.2.1	Security fix: the dedicated fullscreen kiosk URL no longer shows the WordPress admin toolbar to a logged-in administrator viewing it. Previously, anyone at the physical kiosk could see "Howdy, {name}" and a direct link into wp-admin if an admin had ever logged in on that browser. Did not affect the [lexipic_kiosk] shortcode embed (normal WordPress behaviour there), nor the PIN-protected admin panel or REST permission checks.
1.2.0	New: "Approved Kiosk URLs" admin page and a public /allowlist REST endpoint, for vetting which domains the standalone LexiPic Kiosk PWA may connect to in Remote URL mode. The PWA reuses this site's existing kiosk PIN via /pin/verify — no second PIN to manage.
1.1.1	Fix: clicking anywhere on a word card played its audio; only the Play button does now.
1.1.0	Performance: word-card images and audio are now stored as files under wp-content/uploads/lexipic-kiosk/ instead of inline base64 in the database; the set list is fetched with one batched query instead of one per set. Existing sets are converted automatically on upgrade — no action needed, and the .json export/import format is unchanged. Backups are dramatically smaller as a result, and import from older base64-based backups still works.
1.0.0	Initial release.